

AGENT INTELLIGENT DE TESTS

Plan Detaille - Volet Data / IA

Guide complet etape par etape

Stagiaire Bayrem Akka

Specialite Data / Intelligence Artificielle

Encadrant Baha Rahmouni

Duree 4 semaines (20 jours ouvrables)

Projet Agent Tests Fonctionnels Intelligents

Date April 2026

. TABLE DES MATIERES

- 1. Vue d'ensemble du plan
- 2. Etat actuel du projet
- 3. SEMAINE 1 : Formation acceleree (Jours 1-5)
- 4. SEMAINE 2 : Donnees + NLP avance (Jours 6-10)
- 5. SEMAINE 3 : Vision + Execution (Jours 11-15)
- 6. SEMAINE 4 : Reporting + Integration + Demo (Jours 16-20)
- 7. Checklist de validation finale
- 8. Conseils pour la soutenance PFE

1. VUE D'ENSEMBLE DU PLAN

Ce plan couvre les 4 phases du volet Data/IA en 20 jours ouvrables. L'objectif est d'avoir un pipeline fonctionnel de bout en bout : un scenario Gherkin entre, un rapport de test detaille avec captures d'ecran sort. Chaque jour est structure pour que tu comprennes d'abord la theorie (matin) puis que tu pratiques immediatement (apres-midi).

Les 4 phases compressees

Phase 1 - Formation (5 jours) Apprendre NLP, CV, Playwright, Gherkin, FastAPI par la pratique

Phase 2 - Analyse (Integree aux jours 5-6) Collecter scenarios, annoter datasets, definir cas d'usage

Phase 3 - Conception (Integree aux jours 7-8) Concevoir les pipelines NLP et CV pendant l'implementation

Phase 4 - Developpement (10 jours) Implementer, tester, integrer, deployer

STRATEGIE CLE

Ne cherche pas la perfection sur chaque composant. Un pipeline complet qui fonctionne de bout en bout impressionne bien plus qu'un seul module parfait. Pipeline hybride NLP : regex (cas simples, rapide) -> spaCy Matcher (moyen) -> HuggingFace (cas ambigus). Pour YOLO : transfer learning depuis un modele pre-entraine, pas d'entrainement from scratch.

2. ETAT ACTUEL DU PROJET

Le projet est deja scaffolde avec la structure complete. Voici ce qui est deja en place :

- Structure du projet (50+ fichiers, app/api, services, schemas, utils, templates)
- FastAPI avec 9 endpoints REST enregistres et fonctionnels
- Service NLP : pipeline hybride regex + spaCy NER, 12 intentions, extraction d'entites
- Service de mapping semantique : intention -> action Playwright
- Service de generation : Jinja2 + role-based selectors Playwright
- Service de vision : YOLO + fallback contours OpenCV + Tesseract OCR
- Service d'execution : lancement reel Playwright avec screenshots
- Service d'adaptation : popup/cookie, modal, retry, timeout
- Service de reporting : JSON + HTML avec Jinja2
- Pipeline complet : POST /api/ia/pipeline (parse -> generate -> execute -> report)
- 29 tests unitaires (tous passent)
- 3 fichiers .feature exemples (login, search, register)
- Dockerfile + docker-compose.yml prêts

Ce qui reste a faire pour atteindre l'excellence :

- [] Approfondir la comprehension de chaque technologie (NLP, CV, Playwright)
- [] Entrainer un vrai modele YOLO sur un dataset de UI elements
- [] Ajouter HuggingFace CamemBERT pour les cas ambigus
- [] Tester le pipeline end-to-end sur des sites reels
- [] Creer les 5 notebooks Jupyter d'exploration
- [] Constituer le corpus de 50+ scenarios Gherkin annotes
- [] Annoter 200+ screenshots pour YOLO
- [] Docker : test de deploiement complet
- [] Preparer 3 demos pour la soutenance

3. SEMAINE 1 : FORMATION ACCELEREE

Objectif : maitriser les 5 technologies cles par la pratique. Chaque jour suit le schema : 1h theorie -> 3h pratique -> 1h notebook. Tu apprends en construisant directement les briques du projet.

JOUR 1 - NLP avec spaCy - Les fondamentaux (1 jour)

Matin : Theorie NLP (2h)

- Comprendre le pipeline NLP : texte brut -> tokens -> analyse -> entites
- Tokenisation : decouper 'je clique sur le bouton' en [je, clique, sur, le, bouton]
- Lemmatisation : 'cliquent' -> 'cliquer', 'saisis' -> 'saisir'
- POS Tagging : identifier VERB(clique), NOUN(bouton), ADP(sur)
- NER (Named Entity Recognition) : extraire 'nour@email.com' comme EMAIL
- Installer spaCy : `pip install spacy && python -m spacy download fr_core_news_sm`

Après-midi : Pratique (3h)

- Charger le modele : `nlp = spacy.load('fr_core_news_sm')`
- Analyser 10 phrases Gherkin : tokeniser, afficher POS tags, entites
- Utiliser le Matcher spaCy pour detecter des patterns : `[VERB] + 'sur' + [NOUN]`
- Utiliser PhraseMatcher pour detecter 'page de connexion', 'bouton Login'
- Comparer les resultats FR vs EN (`fr_core_news_sm` vs `en_core_web_sm`)

Livable du jour

[] Notebook 01_exploration_gherkin_nlp.ipynb avec les analyses

ASTUCE : Comprendre le Matcher spaCy

Le Matcher cherche des patterns dans les tokens. Ex: `pattern = [{ 'POS': 'VERB' }, { 'LOWER': 'sur' }, { 'POS': 'DET' }, { 'OP': '?' }, { 'POS': 'NOUN' }]` va matcher 'clique sur le bouton' ou 'appuie sur Valider'. C'est plus robuste que les regex car il comprend la grammaire. Dans notre projet, on utilise le Matcher pour les patterns frequents et les regex pour les cas simples (valeurs entre guillemets).

JOUR 2 - NLP avance + HuggingFace Transformers (1 jour)

Matin : HuggingFace + Classification (2h)

- Comprendre les Transformers : modeles pre-entraînés qui comprennent le contexte
- CamemBERT : le BERT français, idéal pour classifier des phrases françaises
- Pipeline HuggingFace : `from transformers import pipeline`
- Classification zero-shot : classer sans entraînement ('cliquer', 'naviguer', etc.)
- Embeddings : représenter une phrase comme un vecteur pour calculer la similarité

Après-midi : Pratique (3h)

- Tester la classification zero-shot sur 20 steps Gherkin
- Comparer : regex simple vs spaCy Matcher vs HuggingFace zero-shot
- Mesurer la precision de chaque approche (% de bonnes classifications)
- Comprendre le pipeline hybride du projet : regex -> spaCy -> Transformer
- Explorer le code dans `app/services/nlp_service.py` et comprendre chaque fonction

Livrable du jour

- [] Notebook 02_entrainement_intentions.ipynb avec comparatif des approches
- [] Tableau de precision : regex (X%), spaCy (Y%), HuggingFace (Z%)

COMPRENDRE LE PIPELINE HYBRIDE

Notre NLPService utilise 3 niveaux : 1) Regex rapides pour les cas evidents (je 'clique' -> cliquer, je 'saisis' -> saisir_texte). 2) spaCy NER pour extraire les entites (emails, noms propres). 3) HuggingFace en fallback pour les phrases ambiguës. Cette approche est optimale : rapide pour 90% des cas, intelligente pour les 10% restants.

JOUR 3 - Computer Vision : OpenCV + OCR + YOLO (1 jour)

Matin : Theorie CV (2h)

- OpenCV : charger une image, convertir en niveaux de gris, seuillage
- Detection de contours : trouver les rectangles (boutons, champs) dans une page
- Tesseract OCR : extraire du texte depuis une image (lire 'Login', 'Email', etc.)
- YOLOv8 : detection d'objets en temps reel, concept de bounding box
- Transfer learning : partir d'un modele pre-entraine et l'adapter a nos classes
- Classes YOLO pour UI : button, input, link, dropdown, checkbox, label, modal, tab

Apres-midi : Pratique (3h)

- Capturer 5 screenshots de sites web (page login, formulaire, e-commerce)
- Appliquer le pre-traitement OpenCV : grayscale, threshold, detection de contours
- Extraire le texte de chaque contour avec Tesseract OCR
- Tester YOLOv8 pre-entraine (detection d'objets generiques) sur un screenshot
- Comprendre le code dans app/services/vision_service.py (YOLO + fallback contours)

Livrable du jour

- [] Notebook 03_yolo_ui_detection.ipynb
- [] 5 screenshots avec contours et texte OCR annotes

COMPRENDRE LE FALLBACK VISION

Notre VisionService fonctionne en 2 modes : 1) Si un modele YOLO entraine existe -> detection precise des elements UI. 2) Sinon -> fallback par detection de contours OpenCV (morphologie + seuillage adaptatif) + OCR. Le fallback est deja fonctionnel ! L'entrainement YOLO ameliorera la precision mais n'est pas bloquant.

JOUR 4 - Playwright + Gherkin BDD (1 jour)

Matin : Gherkin & BDD (1.5h)

- BDD : le comportement attendu est defini AVANT le developpement
- Structure : Feature > Scenario > Steps (Given/When/Then)
- Scenario Outline + Exemples : parametriser les scenarios
- Tags : @smoke, @login pour organiser les tests
- Installer behave : pip install behave
- Ecrire 5 scenarios Gherkin pour differents cas (login, recherche, formulaire)

Après-midi : Playwright (3h)

- Installer : `pip install playwright` & `playwright install chromium`
- Ouvrir une page : `page.goto('https://the-internet.herokuapp.com/login')`
- Selecteurs : CSS (`#username`), XPath, text (`'Login'`), role-based (`get_by_role`)
- Actions : `page.fill('#username', 'tomsmith')`, `page.click('button')`
- Assertions : `expect(locator).to_have_text('Bienvenue')`
- Screenshots : `page.screenshot(path='capture.png')`
- Mode headless vs headed, gestion des timeouts

Mini-projet : Automatiser un login

- [] Script Playwright qui se connecte a `the-internet.herokuapp.com/login`
- [] 5 fichiers `.feature` pour differents scenarios

POURQUOI GET_BY_ROLE EST SUPERIEUR

Notre generateur utilise `page.get_by_role('button', name='Login')` au lieu de `page.click('#btn-login')`. Avantage : si le developpeur change l'ID du bouton, le test continue de fonctionner tant que le texte visible reste 'Login'. C'est exactement ce que recommande Playwright officiellement.

JOUR 5 - FastAPI + Analyse metier (1 jour)

Matin : FastAPI (2h)

- Comprendre les endpoints du projet : GET `/health`, POST `/parse-scenario`, etc.
- Schemas Pydantic : validation automatique des requetes/reponses
- Lire et comprendre `app/main.py`, `app/api/*.py`, `app/schemas/*.py`
- Tester chaque endpoint via Swagger UI (`http://localhost:8000/docs`)
- Upload de fichiers : `UploadFile` pour les screenshots (POST `/detect-ui`)

Après-midi : Analyse metier (3h)

- Collecter 20-30 scenarios Gherkin exemples depuis Internet et projets open-source
- Classifier les patterns : actions (cliquer, saisir), verifications (voir, afficher), navigation
- Construire un dictionnaire : mot Gherkin -> intention -> action Playwright
- Définir les 5 cas d'usage IA (CU1-CU5) et les prioriser (MoSCoW)
- Sauvegarder dans `data/gherkin_samples/` les scenarios collectes

Livrable du jour

- [] 20+ scenarios Gherkin dans `data/gherkin_samples/`
- [] Dictionnaire d'actions documente
- [] Tous les endpoints testes via Swagger

4. SEMAINE 2 : DONNEES + NLP AVANCE

JOUR 6 - Preparation des datasets (1 jour)

Journee complete : Constitution des donnees

- Finaliser 50+ scenarios Gherkin annotes avec intention + entites
- Format d'annotation : {step, intention, entites: [{nom, valeur, type}]}
- Capturer 100+ screenshots de sites web varies (e-commerce, admin, formulaires)
- Sites recommandes : the-internet.herokuapp.com, demoqa.com, saucedemo.com
- Commencer l'annotation sur Roboflow (gratuit) : creer un projet, uploader les images
- Definir les classes : button, input_text, input_password, link, checkbox, dropdown, label, modal
- Annoter au moins 50 images (les plus variees possibles)
- Sauvegarder tout dans data/gherkin_samples/ et data/ui_screenshots/

ASTUCE ROBOFLOW

Roboflow permet d'annoter rapidement avec auto-suggestions. Apres 30 images, il propose des annotations automatiques. Il genere aussi de l'augmentation de donnees (rotation, zoom, luminosite) pour multiplier ton dataset. Exporte en format YOLO (txt).

JOUR 7 - Pipeline NLP ameliore (1 jour)

Matin : Conception du pipeline NLP final (2h)

- Revoir le pipeline actuel dans app/services/nlp_service.py
- Ajouter les patterns spaCy Matcher manquants (formulaires, tableaux, onglets)
- Ameliorer l'extraction d'entites : detecter les XPath, CSS selectors, data-testid
- Gerer les Scenario Outline : remplacer <placeholders> par les valeurs Exemples

Apres-midi : Amelioration du parser Gherkin (3h)

- Ameliorer app/utils/gherkin_parser.py pour parser des .feature complets
- Ajouter le support de Background (steps executes avant chaque scenario)
- Ajouter le parsing de Scenario Outline + substitution Exemples
- Creer un endpoint POST /api/ia/parse-feature qui recoit un fichier .feature
- Tester avec les 3 fichiers .feature existants + les 20 nouveaux

JOUR 8 - Classification d'intentions avancee (1 jour)

Matin : Integration HuggingFace (2h)

- Installer : pip install transformers sentence-transformers
- Tester zero-shot classification sur les steps ambigus
- Integrer comme fallback quand regex + spaCy ne trouvent pas (confiance < 0.5)
- Ajouter Sentence-BERT pour l'embedding semantique (matching labels UI)

Apres-midi : Tests et metriques (3h)

- Creer un dataset de test : 100 steps avec intention correcte connue
- Mesurer precision, rappel, F1-score pour chaque approche

- Documenter les resultats dans le notebook 02_entrainement_intentions.ipynb
- Ajuster les seuils de confiance si necessaire
- Objectif : > 95% de precision sur les steps standard

JOUR 9 - Mapping semantique + Generation amelioree (1 jour)

Matin : Mapping semantique avance (2h)

- Ameliorer le matching : utiliser la similarite cosinus pour fuzzy matching
- Ex: 'Valider la commande' doit matcher le bouton 'Submit Order'
- Integrer text_similarity.py (Levenshtein + TF-IDF) dans le mapping service
- Creer le notebook 04_mapping_semantique.ipynb pour documenter

Apres-midi : Generation Playwright amelioree (3h)

- Ameliorer le template Jinja2 avec gestion des waits intelligents
- Ajouter la capture d'ecran automatique apres chaque step
- Gerer les cas speciaux : iframes, shadow DOM, popups
- Tester la generation sur les 50+ scenarios du corpus

JOUR 10 - Annotation YOLO sprint final (1 jour)

Journee complete : Finaliser les annotations

- Objectif : 200+ images annotees sur Roboflow
- Varier les sources : sites FR, EN, responsive, dark mode, light mode
- Exporter en format YOLOv8 (dossier images/ + labels/ + data.yaml)
- Split : 70% train, 20% validation, 10% test
- Verifier la qualite : pas de bounding boxes trop larges ou manquantes
- Sauvegarder dans data/ui_screenshots/ et data/annotations/

5. SEMAINE 3 : VISION + EXECUTION

JOUR 11 - Entrainement YOLOv8 (1 jour)

Journee complete : Train + Eval

- Installer : `pip install ultralytics`
- Creer le fichier `data.yaml` avec les chemins et classes
- Lancer l'entrainement : `yolo detect train data=data.yaml model=yolov8n.pt epochs=50`
- Utiliser `yolov8n` (nano) pour la rapidite ou `yolov8s` (small) pour la precision
- Surveiller les metriques : `mAP@50`, precision, recall, loss
- Evaluer sur le set de test : `yolo detect val`
- Objectif minimum : `mAP@50 > 70%`
- Sauvegarder le best model dans `trained_models/vision/yolov8_ui_elements.pt`

SI LE MAP EST FAIBLE

1) Augmenter les donnees (Roboflow propose l'augmentation automatique). 2) Re-annoter les images avec les erreurs les plus frequentes. 3) Augmenter les epochs (100-150). 4) Utiliser `yolov8s` au lieu de `yolov8n`. Meme avec un `mAP` de 60%, le fallback OCR+contours compensera.

JOUR 12 - Integration Vision dans le pipeline (1 jour)

Matin : Integrer le modele YOLO (2h)

- Mettre a jour `app/services/vision_service.py` pour charger le nouveau modele
- Tester `_detect_with_yolo()` sur 20+ screenshots
- Affiner le seuil de confiance (`conf=0.4` par default)
- Comparer avec le fallback contours : precision YOLO vs contours

Apres-midi : OCR + Matching (3h)

- Optimiser l'OCR : pre-traitement (resize x2, binarisation Otsu)
- Tester la langue : `lang='fra+eng'` pour le francais et anglais
- Implementer le matching : texte du step Gherkin <-> texte OCR de l'element detecte
- Ex: step dit 'bouton Login' -> YOLO detecte un bouton -> OCR lit 'Login' -> MATCH
- Tester POST `/api/ia/detect-ui` sur 20 screenshots varies

JOUR 13 - Moteur d'execution des tests (1 jour)

Matin : Execution end-to-end (2h)

- Tester l'executor service sur un scenario simple (login the-internet.herokuapp.com)
- Verifier la capture d'ecran a chaque etape
- Gerer les timeouts : `networkidle`, `waitForSelector`
- Implementer les retries en cas d'echec (max 2 tentatives)

Apres-midi : Gestion des cas speciaux (3h)

- Popups de cookies : detection + fermeture automatique

- Modales inattendues : Escape + bouton fermer
- Elements dynamiques : attente intelligente avant interaction
- Tester sur saucedemo.com (e-commerce), demoqa.com (formulaires)
- Documenter les resultats dans notebook 05_generation_playwright.ipynb

JOUR 14 - Pipeline complet end-to-end (1 jour)

Journee : Tout connecter

- Tester POST /api/ia/pipeline avec un scenario Gherkin complet
- Flux : Gherkin -> NLP (intentions+entites) -> Generation Playwright -> Execution -> Rapport
- Scenario 1 : Login sur the-internet.herokuapp.com
- Scenario 2 : Ajouter un produit au panier sur saucedemo.com
- Scenario 3 : Remplir un formulaire sur demoqa.com
- Pour chaque scenario : verifier le script genere, l'execution, les screenshots, le rapport
- Corriger les bugs et edge cases trouves

JOUR 15 - Adaptation dynamique + robustesse (1 jour)

Journee : Rendre l'agent resilient

- Tester l'adaptation service avec differents types d'erreurs
- Simuler : popup cookies, element manquant, timeout, changement de layout
- Implementer le fallback CV dans l'execution : si selecteur CSS echoue -> YOLO + OCR
- Ajouter le retry intelligent : screenshot -> re-detection -> re-essai
- Mesurer le taux de recuperation apres erreur (objectif > 70%)
- Ecrire des tests d'integration pour les cas d'adaptation

6. SEMAINE 4 : REPORTING + INTEGRATION + DEMO

JOUR 16 - Systeme de reporting (1 jour)

Journee : Rapports professionnels

- Verifier que chaque execution genere un rapport HTML propre
- Integrer les screenshots dans le rapport (inline ou liens)
- Ajouter les metriques : duree par step, taux de reussite, navigateur utilise
- Generer un rapport JSON structuree pour le backend .NET
- Tester GET /api/ia/reports/{test_id}
- Optionnel : generer un rapport PDF avec les memes donnees

JOUR 17 - Tests d'integration + qualite (1 jour)

Journee : Robustesse

- Ecrire des tests d'integration pour le pipeline complet
- Tester les edge cases : scenario vide, steps invalides, URL inaccessible
- Tester la performance : temps de response < 500ms pour le parsing NLP
- Verifier la securite : pas d'injection dans les inputs
- Objectif : 40+ tests, tous verts

JOUR 18 - Docker + documentation (1 jour)

Journee : Deploiement

- Tester docker-compose up --build et verifier que tout fonctionne
- S'assurer que les modeles (spaCy, YOLO) sont inclus dans l'image
- Documenter les endpoints dans le README.md
- Ajouter des docstrings sur les fonctions complexes
- Nettoyer le code : supprimer les TODO resolus, formater avec black

JOUR 19 - Preparation de la demo (1 jour)

Journee : Presentations

- Preparer 3 scenarios de demo poils :
 - Demo 1 : Login reussi + verification texte (simple)
 - Demo 2 : Formulaire complet avec validation (intermediaire)
 - Demo 3 : Scenario qui rencontre un popup + adaptation dynamique (avance)
- Pour chaque demo : scenario Gherkin -> pipeline -> rapport HTML
- Finaliser les 5 notebooks Jupyter avec explications
- Preparer des captures d'ecran du Swagger UI et des rapports

JOUR 20 - Polish final + contenu PFE (1 jour)

Journee : Finalisation

- Derniers bug fixes
- Generer les metriques finales pour le rapport PFE :
 - Precision parsing NLP : X% (sur Y scenarios)
 - mAP YOLO : X% (sur Y images)
 - Taux de scripts executables : X%
 - Taux de reussite pipeline end-to-end : X%
 - Temps moyen de traitement par scenario : Xms
- Rediger le contenu IA du rapport PFE : etat de l'art, methodologie, resultats
- Preparer les slides pour la soutenance

7. CHECKLIST DE VALIDATION FINALE

Pipeline NLP (CU1 + CU2)

- ☐ Le parsing detecte correctement Given/When/Then/And/But en FR et EN
- ☐ Les 12 intentions sont classifiees avec une precision > 95%
- ☐ Les entites (valeur, cible, type_element, url, email) sont extraites correctement
- ☐ Le mapping semantique produit des actions Playwright valides
- ☐ Le temps de parsing est < 500ms par scenario

Computer Vision (CU3 + CU4)

- ☐ Le modele YOLO detecte button, input, link, checkbox, dropdown
- ☐ Le mAP@50 est > 70% sur le jeu de test
- ☐ Le Tesseract OCR lit correctement le texte des elements detectes
- ☐ Le fallback contours fonctionne quand YOLO n'est pas disponible
- ☐ Le matching Gherkin text <-> element UI fonctionne

Execution et adaptation (CU5)

- ☐ Le script Playwright genere s'execute sans erreur de syntaxe
- ☐ Les screenshots sont captures a chaque etape importante
- ☐ Les popups de cookies sont detectes et fermes automatiquement
- ☐ Le retry fonctionne en cas d'element non trouve
- ☐ Le pipeline complet (POST /api/ia/pipeline) fonctionne de bout en bout

Reporting et integration

- ☐ Le rapport HTML est genere avec screenshots integres
- ☐ Le rapport JSON est consommable par le backend .NET
- ☐ Tous les 9 endpoints API fonctionnent (testes via Swagger)
- ☐ Docker : docker-compose up --build demarre le service correctement
- ☐ 40+ tests unitaires passent (pytest tests/ -v)

Livrables

- ☐ 5 notebooks Jupyter documentes
- ☐ 50+ scenarios Gherkin annotes
- ☐ 200+ screenshots annotes pour YOLO
- ☐ Modele YOLO entraine (yolov8_ui_elements.pt)
- ☐ 3 demos fonctionnelles de bout en bout
- ☐ README.md complet avec instructions de demarrage

8. CONSEILS POUR LA SOUTENANCE PFE

Structure de la presentation (20 min)

- Introduction (3 min) : problematique des tests fonctionnels + objectifs du projet
- Architecture (3 min) : schema Frontend -> Backend .NET -> Service IA -> DB
- Pipeline NLP (4 min) : demo Gherkin -> intentions -> entites (avec notebook)
- Computer Vision (4 min) : demo screenshot -> YOLO detection -> OCR (avec notebook)
- Demo live (4 min) : pipeline complet sur un scenario reel
- Resultats + perspectives (2 min) : metriques, limites, ameliorations futures

Points forts a mettre en avant

- L'approche hybride NLP (regex + spaCy + Transformers) : pragmatique et efficace
- L'indépendance du code source : l'agent teste l'UI comme un humain (Computer Vision)
- L'adaptation dynamique : l'agent gere les imprevis (popups, changements UI)
- Le pipeline complet automatise : 0 intervention humaine de Gherkin a rapport
- La qualite du code : structure propre, tests unitaires, Docker, documentation

Questions probables du jury

- Q: Pourquoi ne pas utiliser uniquement Selenium/Cypress ?
- -> R: Notre approche est independante du code source, utilise le langage metier
- Q: Comment gerez-vous les changements d'interface ?
- -> R: Adaptation dynamique : re-detection CV, matching semantique, retry intelligent
- Q: Quelles sont les limites de votre approche ?
- -> R: Sites tres dynamiques (SPA complexes), animations, CAPTCHA
- Q: Comment amelioreriez-vous le systeme ?
- -> R: Fine-tuning CamemBERT sur nos donnees, YOLO avec plus de classes, LLM pour generation

CONSEIL FINAL

L'objectif du PFE n'est pas la perfection, c'est la demarche. Montre que tu as compris le probleme, choisi les bonnes approches, mesure les resultats, et propose des ameliorations. Un pipeline qui fonctionne a 85% avec des metriques documentees vaut plus qu'un systeme parfait sans evaluation. Les notebooks Jupyter sont tes meilleurs allies pour montrer ta demarche experimentale.